Module 3

Names, Scopes, and Bindings

Names, Scopes and Bindings

This chapter is all about the names that are used in our programs. Simple as it may seem, there is significant complexity to them.

Definitions

- Identifier - a character string that represents something else
- Name - an identifier (or combination of identifiers) (e.g., x.y.z)
- Binding - an association between two things, such as a name and a memory address.
- Scope - the textual part of a program in which a binding is active.
- Environment - the set of bindings.

Binding Time

The point that a binding is created is called its **binding time**. Binding time can refer to many different points of a programming language's lifetime:

Language Design Time

- Syntax, type-system, semantics

Language Implementation Time

- CFG, input/output, numeric precision

Program development time

- Algorithms, program design, and names

Binding Time Continued

Compile time

- Intra-module bindings and data layout

Link time

- Inter-module bindings and whole program layout

Load time

- Virtual Memory addresses

Run time

- hardware addresses and variable values

“Styles” of binding

Static binding refers to a binding made before run time.

Dynamic binding refers to a binding created at or during run time.

Static binding allows for more optimization, and dynamic binding leads to more flexibility

These can be roughly analogous to compiled and interpreted strategies for translation. e.g.:

A compiler can analyze and layout all of its global variables and generate efficient code to access them, only one time

An interpreter needs to do that every time the program is executed.

Scope Overview

A key feature of all programming languages is the ability to **name** data (relying on this is one thing that makes the Scheme assignment so different!).

Names are convenient because they allow a programmer to name some data, and that name can be used instead of a memory location.

The **scope rules** of a programming language are what determine name and variable bindings.

Examples of different scope rules:

Scope in Go, C

Scope in Bash

This is a nuanced topic, and there are many things that influence what scope rules are chosen over others.

Note: not all data is named, such as data that is referenced by pointers.

Lifetime I

Names and Bindings have specific lifetimes in our programs, bounded by specific events.

- Creation and destruction of objects
- Creation and destruction of bindings
- Deactivation and reactivation of bindings that may not be usable temporarily
- References to variables, subroutines, types etc. All use bindings

Lifetime II

Binding extent - the time between its creation and destruction, AKA its lifetime.

Objects also have lifetime, but this is not always the same as the lifetime of a binding.

An object is any computer construct in memory (ints, Cats, chars etc.)

Reflection: when is object lifetime different from binding lifetime?

Lifetime III

An object becomes **garbage** if it outlives all of its bindings.

A binding that outlives its object is called a **dangling reference**.

The literal textual area that a binding is active is called its **scope**.

Object Lifetime and Storage Lifetime

Memory I

The concepts in this module are strongly coupled with how computer memory is organized for a program that is executing.

When a variable is declared and assigned, ‘where’ does it actually live?

Random Access Memory is the only option. How is that organized?

How about a variable that may or may not be created based on the execution of a program?

Global variables?

Dynamic structures such as linked lists?

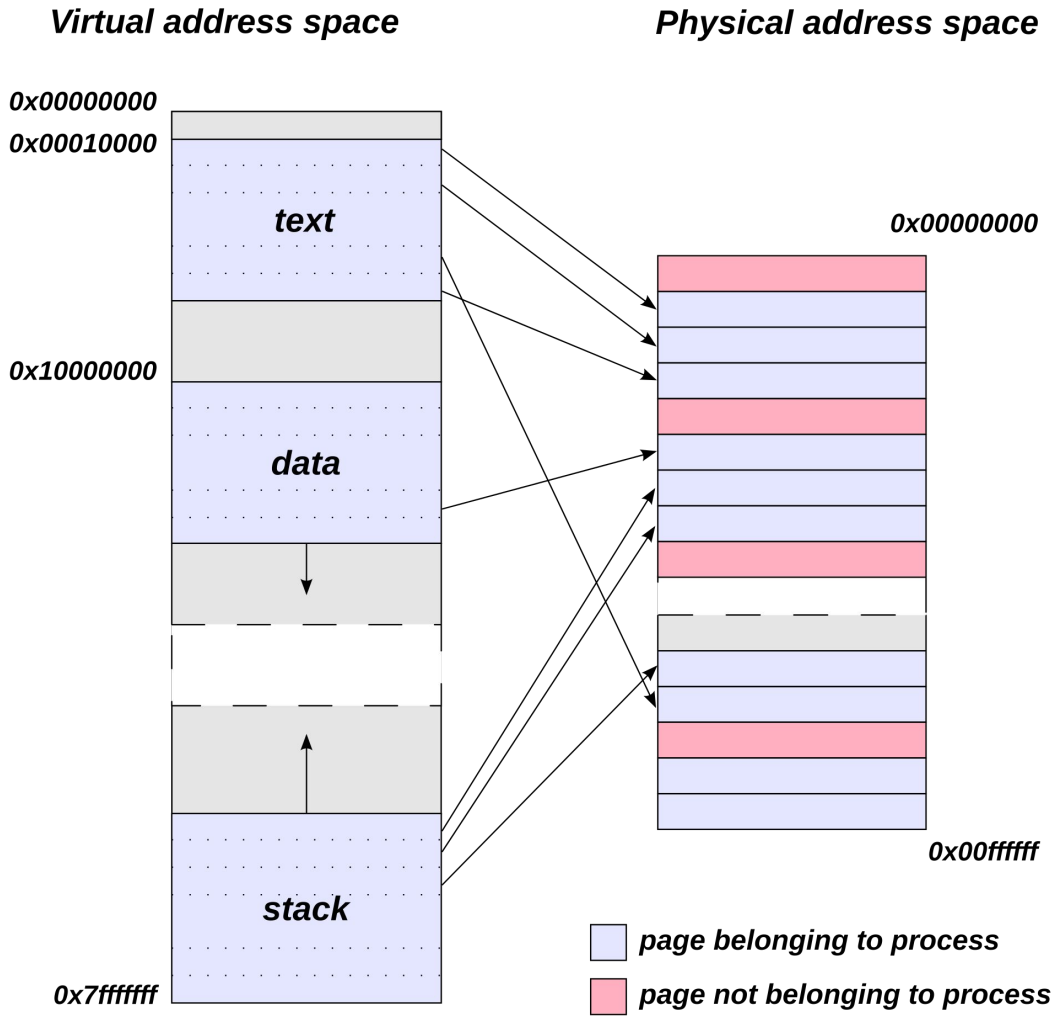
Memory II

Programs are granted 3 types of memory allocation that can be used during their execution:

1. **Static/text/data** - Objects are given an absolute address that is retained throughout the program's execution.
2. **Stack** - objects are allocated and deallocated in last-in, first out order, usually following subroutine calls and returns.
3. **Heap** - objects may be allocated and deallocated arbitrarily. Managing their storage is sophisticated and expensive.

Virtual Addressing

Operating Systems provide a private playground for each program that is executing



Virtual Addressing

Operating Systems provide a private playground for each program that is executing, and it tricks the programs into thinking they have the entire space to themselves. (Much more on this in Operating Systems).

In a 64-bit linux environment (x86_64), 8 bytes are used to address any byte in a 16-exabyte region

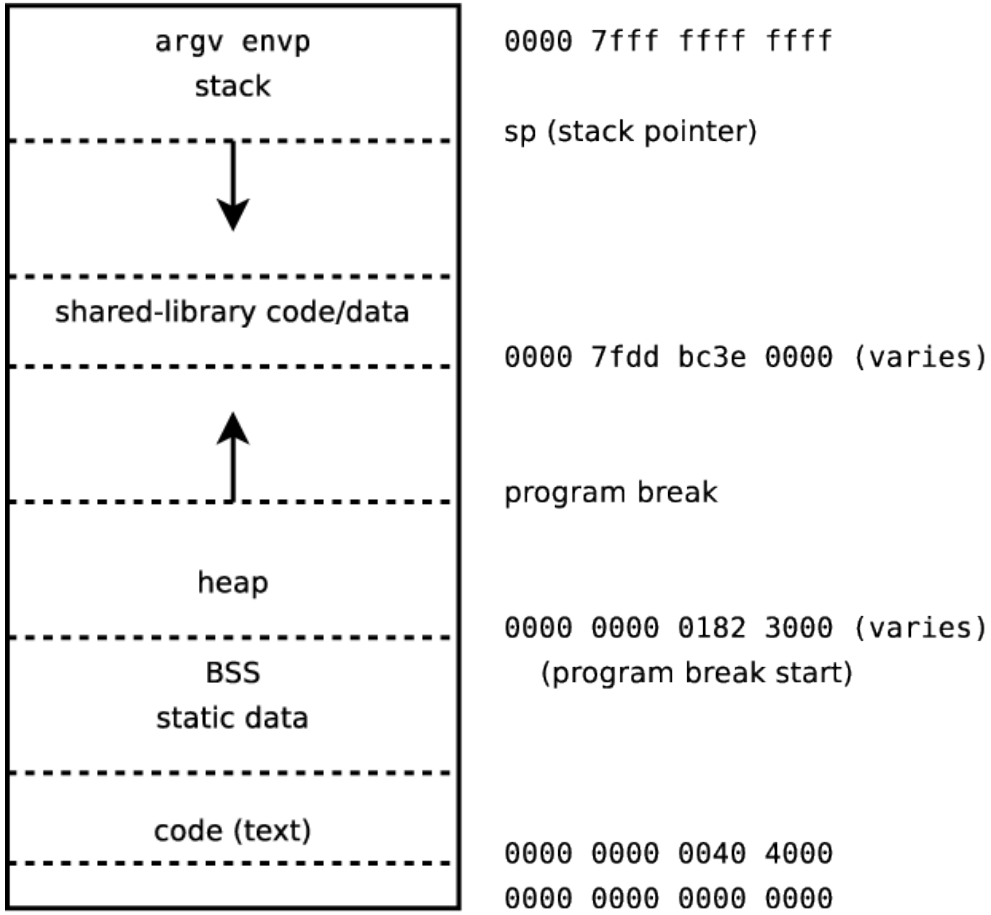
(e.g. 0 - `7fff ffff ffff ffff`).

The Linux Operating system defines the user-space; the lower 256-terabytes are able to be used by user programs.

In class example:

```
pmap -x ./sleep
```

Linux x86_64 Process Memory
User Region



Static Allocation

Since programs are memory, they need to be loaded into memory to run.

When a compiler generates machine code, that code is the first thing loaded into the virtual memory. This is the text and static allocation.

Static Allocation Continued

Other statically allocated items:

- Machine code (translated from the source program)
- globals
- `static` or `own` variables
- explicit constants (including strings, sets, enums, etc)
- scalars may be stored in the instructions themselves if there is room (e.g. 4)

Example: Inspect stack-traversal-simple with od and vim

Stack Allocation

Stack Allocation

Many dynamic activities our programs do end up on the “stack”. Essentially, anything related to a subroutine call goes there.

Discuss: What needs to be stored on the stack when a function is called?

```
struct Person increment_age(struct Person p){

    int myage = p.age;

    p.age = p.age + 1;

    return p;

}
```

Each time a function is called, the data required for the subroutine is kept on the stack and kept until the function returns.

Recursion causes program memory growth!

If recursive functions are improperly implemented and cause infinite recursion, the result is a stack overflow.

If your program encounters a bug and you want to learn about why, most languages print out a nice dialog called a **stack trace**. This is a description of the **call/return stack** that the program had when it was executing before the errors.

Even more, debugging programs allow you to inspect the call stack, and even to see variable scopes and other details of the execution.

Reflection

Why use a stack? Why not a queue? Or Tree? Or Hashtable?

Memory management of the stack

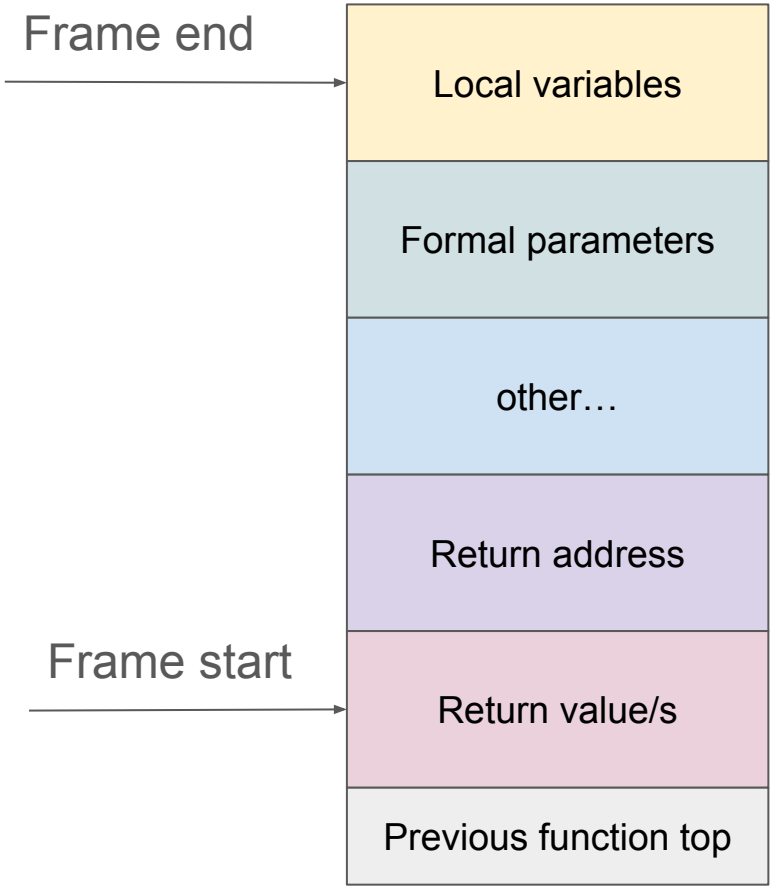
Everytime a subroutine is called, and everytime a “block” is entered, the data pushed onto the stack is grouped into a **frame**.

Organization of this frame is an important aspect of any translator’s design.

Stack Management

Typically, the following are included:

- Local variables
- Formal parameters
- Saved registers
- Scope information
- Return address
- Return values



Stack Management

A subroutines code (which is part of the **static allocation**) accesses stack data via offsets from the beginning of the frame. These offsets can be computed at compile time.

It is not possible to statically determine absolute stack offsets statically.

A frame is bracketed by a **frame pointer**, and the **stack pointer**.

Example: stack-traversal-simple.c

Suppose the following occurs:

A() calls B()

B() calls B() recursively

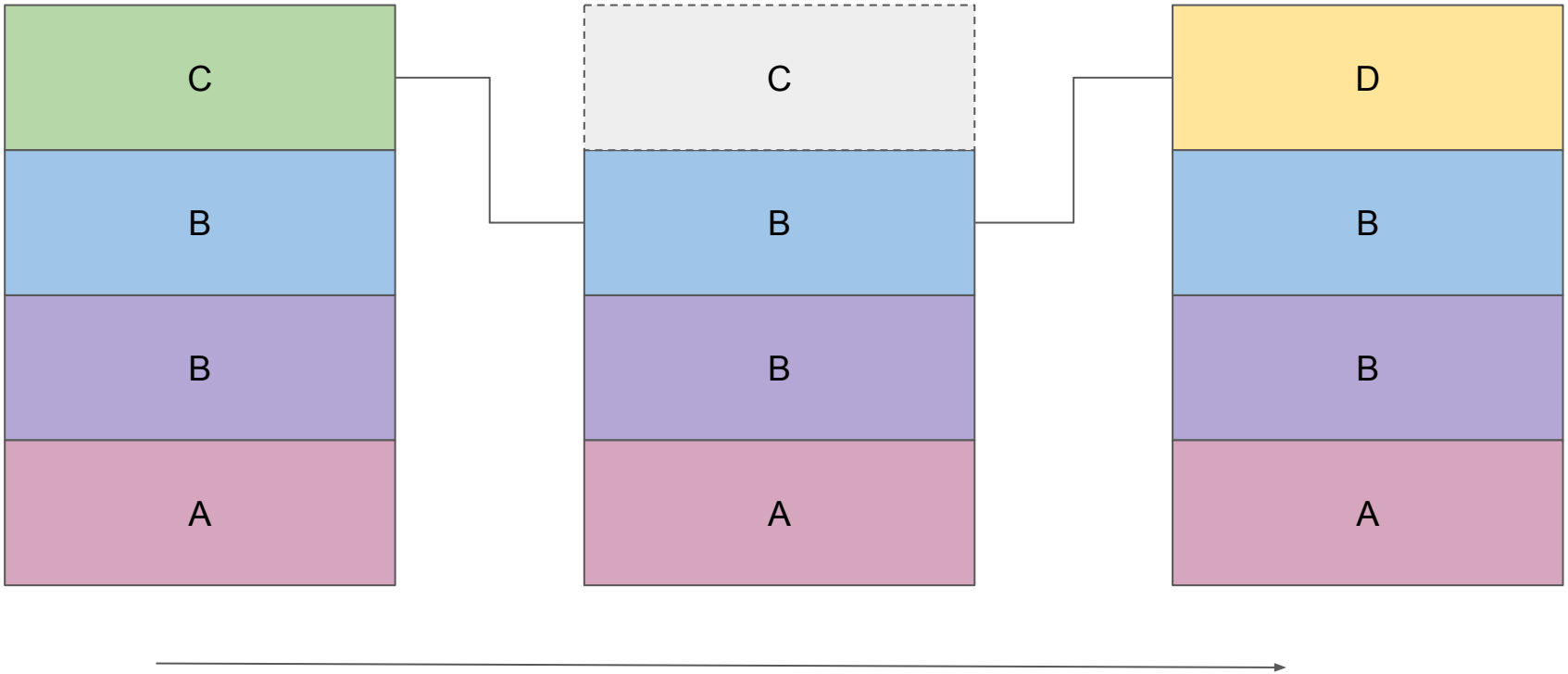
B() then calls C()

C returns to B and calls D()

What does the call/return stack look like?

Diagram

C's space is reclaimed



Reflection

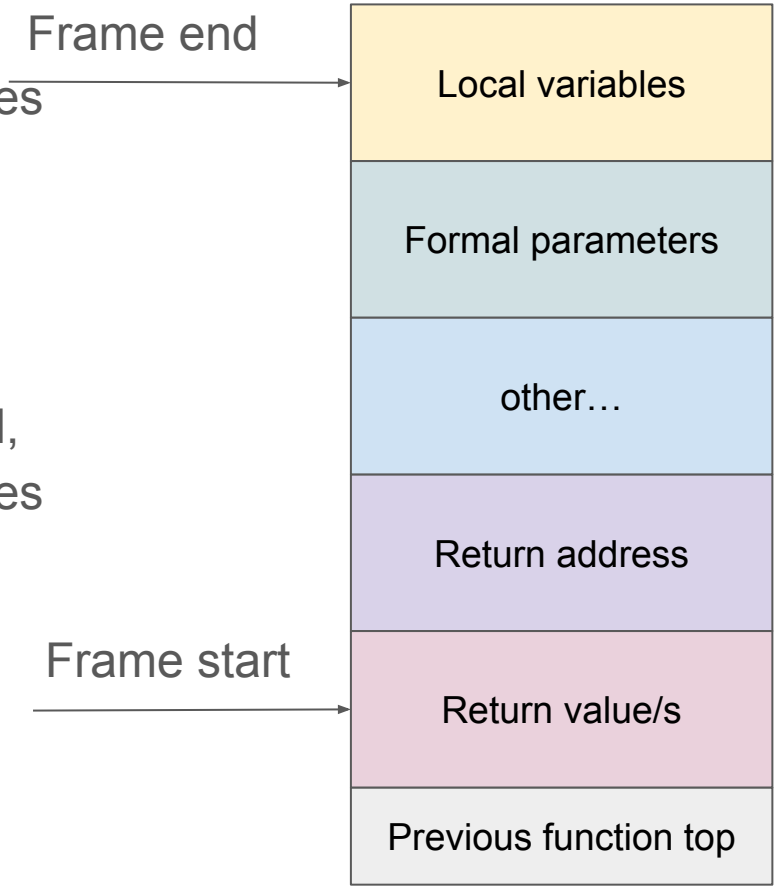
Why use a stack? Why not a queue? Or Tree? Or Hashtable?

- Natural space for recursive procedures
- Reuse space

Calling Sequence I

Allocating and deallocating stack frames is the responsibility of both the **caller** and **callee** of a function call.

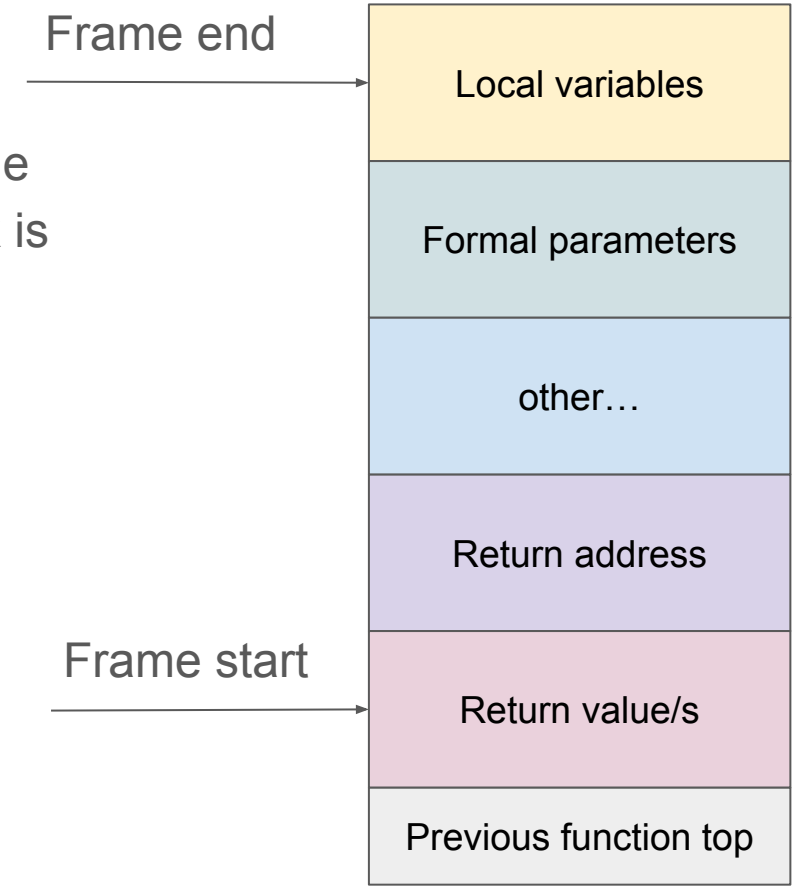
The **callee** is the function that is called, and the **caller** is the function that makes the function call.



Calling Sequence II

The caller performs the **calling sequence** (which includes pushing the actual parameters). The callee's work is known as the **prologue** (such as allocating space for local variables).

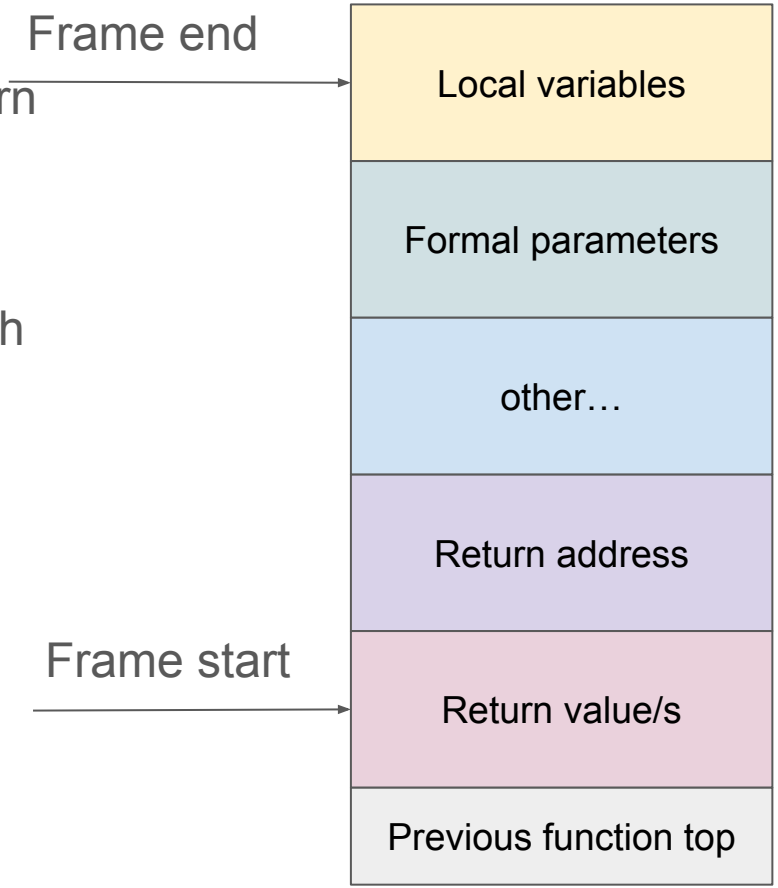
When the callee is done operating, it finishes with the **epilogue** (e.g., deallocate space for the stack frame.



Calling Sequence III

Finally, the **caller** can retrieve the return values from just above the top of the stack.

- Space is saved by putting as much in the prologue and epilogue as possible



Heap Allocation

The Heap

Memory allocation on the **heap** takes place when new objects are created that need to be persistent beyond a function call.

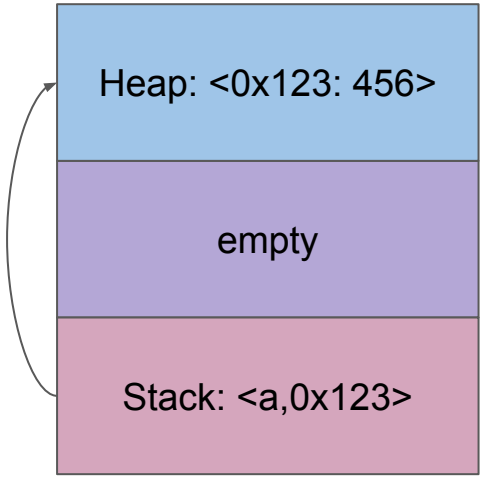
The heap grows “downward” toward higher memory addresses in most cases.

In Java, this is typically the result of the `new` operator, and in C, the result of a call to `malloc`. Go provides `make` and `new`. Go will occasionally put local function variables on the heap.

How many objects are created with a call to `malloc`?

malloc()

```
int *a = (int *)malloc(ARRAY_SIZE * sizeof(int));
```



Heap storage-management-algorithms

Many languages include a **storage-management-algorithm** to manage space on the heap.

Some languages provide no automatic management of the heap space.

Does `new` in java?

Does `malloc`? Why or why not?

Fragmentation

As usual, the principal concerns of these algorithms are *time* and *space*.

More space is required to use less time.

More time is required to use less space.

There are practical considerations as well.

This possibly results in unexpected behavior.

Fragmentation

Internal fragmentation occurs when the algorithm allocates a block that is larger than required to hold a specific object.

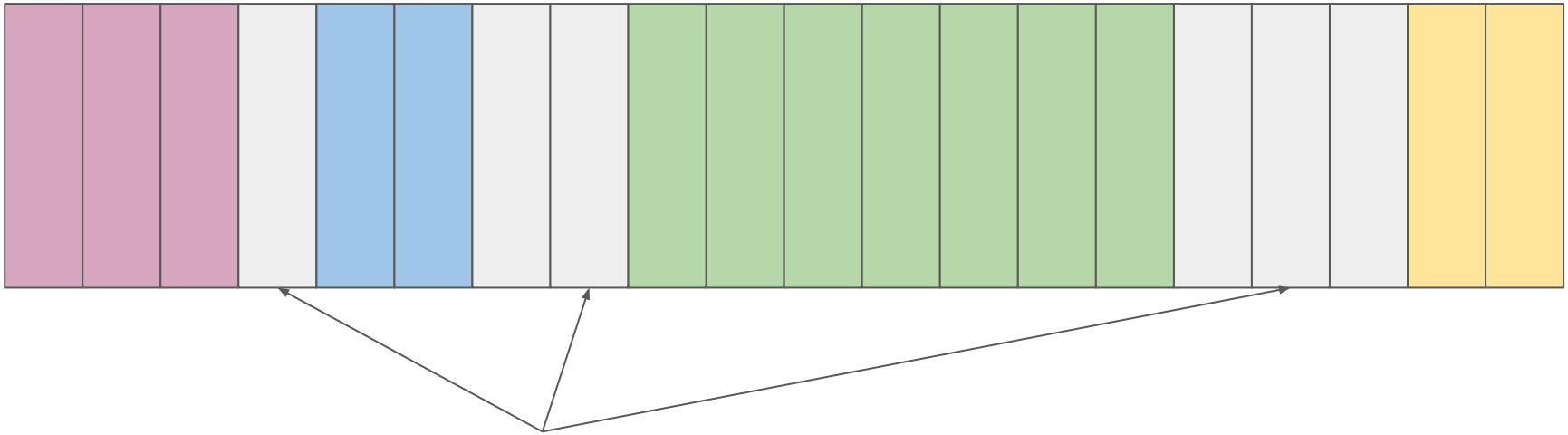
Example: fragmentation.go

External fragmentation occurs when the blocks that have been assigned to active objects such that reallocating the space between them is difficult or impossible.

Example: fragmentation.c

Heap

Items that are **freed** from the heap (either by automatic garbage collection or a call to `free()` in C) can leave empty space in between other heap objects… This can accumulate as the problem of **fragmentation** on the heap.



Heap - Garbage Collection

Garbage collection is a common and sought after feature in many programming languages.

Why garbage collection? Why not?

Traditional arguments **against** of garbage collection are compiler implementation simplicity and program execution speed.

Even simple implementations of garbage collection add significantly to the complexity of the language and to program runtime.

Garbage Collection

Common storage-management-algorithms use a **free-list** to provide heap blocks upon request. This generally means allocating memory has an O(n) overhead. Two mechanisms exist to bring this closer to constant time overhead:

Buddy system *O(lg n)*

Fibonacci heap

These will be explored in detail in your course on Operating Systems.

Scope Rules

Scope Rules I

The textual region of the program in which a binding is **active** is that binding's scope.

In general, there are two styles of scope that our programming languages have.

Static Scope rules - scope.go

Dynamic Scope rules - scope.sh

Scope Rules II

Typically, a scope is the body of a module, class, subroutine, or structured control-flow statement, sometimes called a block.

Is static scope different than static binding?

A static binding is one made by `const` or `final` modifiers

Statically scoped languages support dynamic bindings, like the pointer to the changing head of a linked list.

Referencing Environment I

The set of active bindings at any given time in the program's execution is the **referencing environment**.

The rules of the referencing environment determine the rules of scope.

Scope rules can be **static** or **dynamic**.

A referencing environment is generally provided as a **collection of bindings** that can be examined in specific order to find the correct binding for a given name.

For very simple scoping rules, a referencing environment is an associative datastructure containing variable names mapped to variable values.

For static rules like blocks, a referencing environment is a stack of maps of variable names to variable values.

Referencing Environment II

How is a referencing environment utilized by a programming language?

1. A declaration is seen: The declaration's name is stored in the environment
2. An assignment is made: The environment stores a value for the identifier

An assignment *may* result in an error if a name is not yet declared

Java? Bash? Javascript? Go?

This error can be detected statically or dynamically

Some languages treat an assignment may serve as a declaration for the name

Referencing Environment III

Draw pictures to visualize:

Static Scope rules - scope.go

Dynamic Scope rules - scope.sh

Static (Lexical) Scope

Static scope is also referred to as Lexical scope. With static scope rules, a scope is defined in terms of the physical (lexical) structure of the program.

Basically, the current binding for a given name is found in the matching declaration whose block most closely surrounds a given point in the program.

Static Scope Key Points

- The determination of scopes can be made by the compiler
- All bindings for identifiers can be resolved by examining the program text
- Typically, we choose the most recent, active binding made *at compile time*
- Most compiled languages, C, Go and Java included, employ static scope rules
- Block scope creates new scopes between certain delimiters (typically `{ }`).

To resolve a reference to an identifier, the referencing environment examines the local scope and statically enclosing scopes until a binding is found.

Example: block-local.go

A Practical Example

A practical use of block scope:

```
{
    int temp = a;
    a = b;
    b = temp;
}
```

`temp` is not accessible later in the program, and its stack memory *could* be reclaimed.

What about that “static” keyword

In Java, `static` refers to fields associated with the class, but not any particular instance of the class.

In C, `static` refers to a variable whose lifetime encompasses the entire runtime of the program, even when the name is out of scope.

Their inherent difference is the “globalness” of each.

See Scopes.java. See static.c

Did you know this?

PLs allow us to create functions.

Where can functions be created?

scope.js

nonlocal.js

Methods of Finding Variables

Some programming languages allow subroutines to be nested within each other (like Scheme, Javascript, Go and others)

In this case, the job of finding the right identifier becomes challenging.

One solution is to use static links in each stack frame to allow the current frame to find a variable that is referenced.

Finding the variable value involves traversing the list of links.

Another solution uses “displays”, which keeps an array of names and scopes. Finding a variable value involves a O(1) search.

Static Links

During compilation, compute the static nesting depth of each declaration (d) and depth of each reference (r).

During execution, maintain an entry in each frame, pointing to the nearest frame with the previous nesting depth.

To get the reference, follow `k = r - d` static links, and access that frame.

Example: [nonlocal.js]

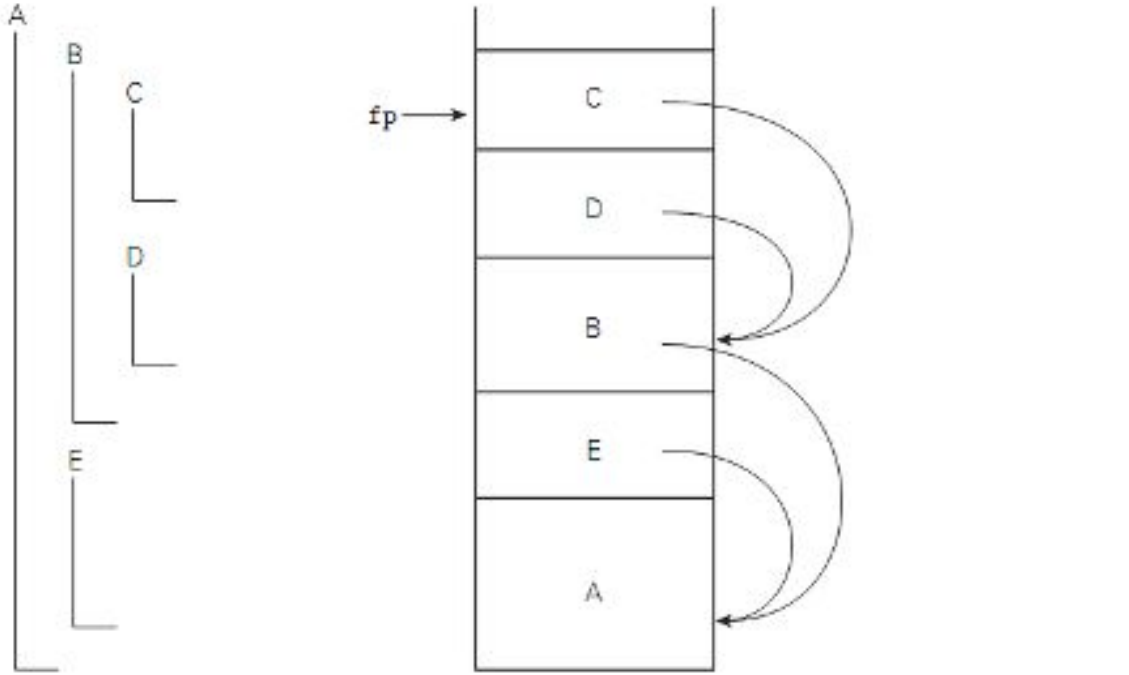


Figure 3.5 Static chains. Subroutines A, B, C, D, and E are nested as shown on the left. If the sequence of nested calls at run time is A, E, B, D, and C, then the static links in the stack will look as shown on the right. The code for subroutine C can find local objects at known offsets from the frame pointer. It can find local objects of the surrounding scope, B, by dereferencing its static chain once and then applying an offset. It can find local objects in B's surrounding scope, A, by dereferencing its static chain twice and then applying an offset.

Why Nested Subroutines?

- Allow private inner subroutines
- Can lead to more compact code
- Can further hide data in closures

Example: sqrt.js

Example: sqrt.scm

Higher Order Functions

When a function is

- Assigned to a variable
- Passed to a function
- Returned from a function
- Constructed from data (Scheme!)

What set of bindings does it get?

Binding Rules for higher order functions

Many programming languages have this feature.

The translator needs to decide if the referencing environment of the subroutine is established when the subroutine was created, or when it is used.

Deep binding - Subroutine uses bindings established when it was created.

Shallow binding - Subroutine uses bindings established when it is used.

Higher Order Examples

What happens when we create a function?

higher-order.js

What needs to happen for `f2()` to print 3 ?

What needs to happen for `f2()` to print 2 ?

Binding time

If bindings are fixed early, when the function is assigned, it's called **deep binding**.

If bindings are fixed late, when the function is called, it's called **shallow binding**.

Neither is always best. Deep binding is the most commonly preferred

Static scope rules tend to result in deep binding

Dynamic scope rules tend to result in shallow binding

Closure

When a function is used in the aforementioned scenarios, it is bundled with its own referencing environment. This self-contained object is called a **closure**.

A closure is how **deep binding** is implemented.

Closures can be used to represent instances of object classes, make data private and more.

Closure II

Examples:

1. Go
2. Scheme
3. Javascript
4. Bash

Closures in Object Oriented Programming

Javascript has a model for inheritance (prototypal)

Objects can have data members and functions for accessing them (methods).

prototype.js

This is not very interesting though. `getValue()` is unnecessary because `value` is not hidden.

We can hide the instance values inside of a closure though.

prototype-closure.js

closure-car.js

Scope Odds and Ends

Declaration Order

Names are active for the scope they are declared in

Does that mean the whole scope? What exceptions are there to that?

- Forward.java
- forward.js
- flip-flop.c

Forward references are generally static semantic errors

They are also necessary to support mutual recursion

Java allows them for methods to assist the programmer

Dynamic Scope

Dynamic scope rules are determined not by the program's structure, but by the execution path of the program.

Dynamic scope is typically used in interpreted PLs with no compile-time type checking.

Dynamic scope rules don’t support closures generally, making bindings **shallow**

Implementing dynamic scope rules uses a stack of referencing frames, called and A-list (for association list)

A binding is pushed when it is seen at runtime. It is popped when its scope is left.

Finding a binding is done by searching the list top to bottom

Dynamic Scope in statically scoped languages

Languages with static scope can have exceptions where dynamic scope like behavior is required.

Consider these examples:

DynamicScope.java

dynamic-scope.c

Meaning of Names in Scope

Definitions:

Shadowing: The same name refers to different objects depending on scope.

Overriding: The same name refers to different objects depending on class.

Aliasing: The same object is referred to by more than one name.

Overloading: The same name refers to different objects depending on type.

Polymorphism: Having more than one form. A single symbol has more than one type.

2 types of polymorphism

1. Subtype polymorphism
2. Parametric polymorphism

Definitions:

Namespace: a set of names that are used to identify and refer to objects of various kinds.

Module: A namespace unit.

Package: AKA Module.

Aliasing

Sometimes, multiple names can refer to the same object

```
//Alias the Names object
Names obj = new Names(1);
Names alias = obj;
alias.setA(6);
System.out.println(obj.getA()); //prints 6
```

Overloading

```
public class Overloaded {
    private static void f(int i) {
        System.out.println("int: " + i);
    }
    private static void f(double d) {
        System.out.println("double: " + d);
    }
    public static void main(String[] args) {
        f(123);
        f(1.23);
    }
}
```

Overloading Support

Overloading is common in Java but not supported in other languages

Many implementers believe this feature creates hard to understand code

C does not support overloading. C also does not support namespacing. Functions names need to be unique globally

Go does not support overloading. Go does support package namespacing.

Functions names need to be unique at a package level

Java supports overloading and package namespacing. Method names need not be unique

Overloading vs. Overriding

Overloading can sometimes be confused with Overriding.

Consider `equals()` in java, and two classes. CatDog.java

Overloading shares the *same name* with *different objects* (equals() method in this case

Overriding points an existing object to another

This is accomplished in java with inheritance

This can be confusing in Java because all classes have `Object` in their inheritance chain

Can variables be overloaded? Printer.java

Some languages allow overloading operators (i.e. + or <<) (python examples)

Polymorphism

Sometimes, a name can be more than one **form** of object.

- One object with *multiple implementations* is said to be polymorphic
- One object that can be instantiated at compile time in more than one way is said to be Generic

Have you ever wondered how polymorphism implemented?

We will cover this in Module 5

Subtype Polymorphism

Subtype polymorphism gives one type the properties of another. One type needs to be derived from another somehow.

The properties of the super type which are passed to a subtype are said to be inherited

Subtype polymorphism needs to allow overriding to customize a subtype's behavior

The relationship between classes and subclasses and their application is called variance

Types can be applied covariantly or contravariantly

A fantastic example is the Fact abstract class in the interpreter

Subtype Polymorphism II

So far, you are familiar with Java's version of this.

Go uses a compositional model called embedding and interfaces.

C++ has multiple inheritance.

Javascript uses a prototypal model, which should be new to you.

Javascript also supports "classical" syntax for inheritance.

Functional objects in Javascript can also inherit from other "parents"

:eye_roll:

Generics - Parametric Polymorphism I

Polymorphism can be achieved through parameterization

An abstract symbol is used to define the program's type

The abstract symbol has no definition, but may have upper or lower bounds

The abstract symbol can be legally substituted for any type

The type is therefore said to be **generic**

Parametrically polymorphic definitions are uniform: they behave identically regardless of the type they are instantiated with.

Generics - Parametric Polymorphism II

You already have experience with Generic classes in Java.

Here is something new to you: Generic functions/methods. (probably)

Generic methods introduce their own type parameters. They can be used regardless of if the class has a generic definition:

```
public <T> void compute(T parameter) {...}
```

Arrays.asList is good example of this and a some other powerful language features

generic-function.go implements this in Golang

Generics - Parametric Polymorphism III

Many languages have generics.

- Go “recently” added generics. Using them has similar syntax to Java
- Go generics also allow you to provide bounds, but with logical operators

Generic types can be bounded

The bounds can apply for subtypes, or occasionally for supertypes

This corresponds to covariance and contravariance of types

Module 5 will offer a deeper look

~ *fin* ~

Binding of Referencing Environments

Static Scope

Each scope can be represented by a map from identifiers to declarations.

Nested scopes can be represented by a **stack** of scope maps.

This symbol table represents a referencing environment. This is all done at compile time.

Dynamic Scope

Dynamic scope has essentially two options, with a classic computer science tradeoff.

Slow access, fast calls: Bindings can be represented by a list of pairs (i.e., an association list). Each pair is an identifier and its declaration. The a-list is maintained in a stack-like way. This is all done at run time.

Fast access, slow calls: Instead we could maintain a table, indexed by identifier. Each entry is a stack of declarations for that identifier.

Summary: For a compiler

For **static scope**: At compile time, use a **stack of maps** (each map is for one scope). Also, Generate code that manages stack frames at runtime.

For **dynamic scope**: While interpreting, maintain an association list (a-list) of bindings. At runtime, this list functions like a stack of the last seen binding for each name.

Macro Expansion

Macros work by manipulating tokens in the programming language, typically according to some specification that is also written in the source files.

A macro is defined (optionally with arguments) and can be referenced later, causing the macro to ‘expand’ (that is, to replace its reference).

gcc -E will produce a source file after the macro's have been expanded, which lets us figure out which x is referred to inside the macro.

Here is another example, this time with parameters, to perform our block-scoped swap from earlier:

```
#define SWAP(a, b) {int t = (a); (a = b); (b) = t;}
```

Syntax Macros in Scheme